학습 교안

하네스 엔지니어링 입문 0단계

AI 에이전트에게 맡기되, 코드 없이 배우는 HTML·CSS 명함 페이지 한 장

OpenCode · OpenRouter · 브라우저

0장 준비 + 1~6장 · 프로그래밍 언어 없이 진행하는 실습

강사 김대철

작성일 2026.11.12

실습 환경 Windows + Git + OpenCode (Python 불필요)

실습 저장소 https://github.com/sinaboro/harness-beginner-tutorial

교안의 명령과 테스트 결과는 Windows PowerShell 에서 실제로 실행해 얻은 것입니다.
AI 의 답변은 실행할 때마다 달라지므로, 교안에는 답변 대신 확인 방법을 실었습니다.

목차

과정 안내 4

0단계 준비하기 (도커) 5

1. Docker Desktop 설치 (Windows) 5
2. 저장소 받기와 설정 파일 (Windows) 5
3. 작업 환경 만들기 (Windows, 처음 한 번) 5
4. 컨테이너 들어가기 (Windows) 6
5. OpenRouter 연결 (컨테이너 안) 6
6. 작업 폴더 확인 6

확인 체크리스트 6

자주 만나는 오류 7

0단계 1장. AI 와 하네스 체험 8

학습 목표 8

1. 개념 8
2. 따라 하기 8
3. 확인하기 10
4. 자주 만나는 오류 10
5. 확인 문제 10

0단계 2장. 말로 된 요구사항 쓰기 11

학습 목표 11

1. 개념 — 요구사항은 AI 와의 약속 11
2. 따라 하기 11
3. 확인하기 12
4. 자주 만나는 오류 12
5. 확인 문제 12

0단계 3장. 작업 지시서 쓰기 13

학습 목표 13

1. 개념 — 일을 나눠 주는 쪽지 13
2. 따라 하기 13
3. 확인하기 15
4. 좋은 지시와 나쁜 지시 15
5. 확인 문제 15

0단계 4장. 규칙과 권한 세우기 16

학습 목표 16

1. 개념 16

2. 따라 하기 . . . 16
3. 확인하기 . . . 18
4. 자주 만나는 오류 . . . 18
5. 확인 문제 . . . 18

0단계 5장. 체크리스트로 확인하고 저장하기 . . . 19

학습 목표 . . . 19
1. 개념 — 말과 확인은 다르다 . . . 19
2. 따라 하기 . . . 19
3. 확인하기 . . . 20
4. 자주 만나는 오류 . . . 21
5. 확인 문제 . . . 21

0단계 6장. 미니 프로젝트: 자기소개 명함 페이지 . . . 22

학습 목표 . . . 22
1. 개념 — 지금까지의 고리 . . . 22
2. 따라 하기 . . . 22
3. 확인하기 . . . 24
4. 막혔을 때 . . . 24
5. 확인 문제 . . . 24
다음 단계 . . . 25

부록. 템플릿과 예제 . . . 26

A-1. 명함 요구사항 (2장) . . . 26
A-2. 명함 작업 지시서 (3장·6장) . . . 27
A-3. 규칙 문서 빈칸 틀 (4장) . . . 28
A-4. 권한 설정 (4장) . . . 29
A-5. 확인 체크리스트 (5장) . . . 30

과정 안내

0단계는 프로그래밍 언어를 배우지 않고, AI 에이전트에게 맡기면서 하네스의 원리를 익히는 과정입니다. 결과물은 HTML 과 CSS 로 만든 자기소개 명함 페이지 한 장입니다. 1단계(Python 과 자동 테스트)로 넘어가기 전에 이 과정을 먼저 합니다.

0단계 준비하기 (도커)

시간	1시간 30분 (처음 한 번)
준비물	Windows 10·11, Docker Desktop, Git for Windows, VS Code, 인터넷, OpenRouter 계정
결과	도커 컨테이너 안에서 OpenCode 가 OpenRouter 에 연결되고, 작업 폴더가 Windows 와 연결된다

0단계는 프로그래밍 언어를 배우지 않습니다. 실행 환경은 **도커 컨테이너**입니다. PC 에 Python 이나 OpenCode 를 따로 설치하지 않습니다.

> **역할 나누기**
> - **Windows 쪽(PowerShell)**: 설치하고, 저장소를 받고, 컨테이너를 켠다. 명령 블록에 `[Windows]` 라고 표시합니다.
> - **컨테이너 쪽(bash)**: 교안의 실습 명령을 친다. 명령 블록에 `[컨테이너]` 라고 표시합니다.
> - **편집과 확인**: Windows 의 VS Code 에서 파일을 고치고, 브라우저로 결과를 봅니다.

1. Docker Desktop 설치 (Windows)

1. https://www.docker.com/products/docker-desktop/ 에서 받아 설치합니다.
2. 설치 중 **Use WSL 2 instead of Hyper-V** 가 보이면 체크된 채로 둡니다.
3. PC 를 재시작하고 Docker Desktop 을 실행합니다. 왼쪽 아래가 **Engine running** 이 될 때까지 기다립니다.

[Windows] (PowerShell)

```
docker version
```

Client 와 Server 두 줄이 모두 나오면 됩니다. Server 가 없으면 Docker Desktop 이 덜 켜진 것입니다.

> WSL 오류가 나면 관리자 PowerShell 에서 `wsl --update` 를 실행하고 재부팅합니다. BIOS 의 가상화(VT-x 또는 AMD-V)가 꺼져 있으면 켭니다.

2. 저장소 받기와 설정 파일 (Windows)

[Windows] (PowerShell)

```
git clone https://github.com/sinaboro/harness-beginner-tutorial C:\harness-beginner-tutorial
New-Item -ItemType Directory C:\harness-lab
cd C:\harness-beginner-tutorial\docker
Copy-Item .env.example .env
```

VS Code 로 `C:\harness-beginner-tutorial\docker\.env` 를 열어 `GIT_NAME` 과 `GIT_EMAIL` 을 본인 것으로 바꾸고 저장합니다.

3. 작업 환경 만들기 (Windows, 처음 한 번)

[Windows] (PowerShell)

```
cd C:\harness-beginner-tutorial\docker
docker compose build
```

처음에는 몇 분 걸립니다. 마지막에 `Built` 또는 `Successfully` 가 보이면 됩니다.

4. 컨테이너 들어가기 (Windows)

[Windows] (PowerShell)

```
docker compose run --rm workspace
```

프롬프트가 `root@…:/workspace#` 로 바뀌면 컨테이너 안입니다. 이제 아래는 **컨테이너 안(bash)** 에서 칩니다.

[컨테이너]

```
pwd
ls /tutorial/templates/stage0
git --version
python --version
opencode --version
```

`/workspace` 는 Windows 의 `C:\harness-lab` 과 같은 폴더입니다. `/tutorial` 에는 교안 저장소가 보입니다(읽기 전용).

5. OpenRouter 연결 (컨테이너 안)

OpenRouter 키 만들기는 https://openrouter.ai 에서 합니다. 키는 비밀번호와 같으니 채팅이나 화면 공유에 보여주지 않습니다.

[컨테이너]

```
opencode
```

입력 칸에 `/connect` 를 치고 **OpenRouter** 를 고른 뒤 키를 붙여 넣습니다. `/models` 로 목록이 나오면 `/exit` 로 나옵니다.

> 연결 정보는 도커 볼륨에 남습니다. 다음에 `docker compose run --rm workspace` 로 다시 들어가도 키를 다시 넣을 필요가 없습니다.

6. 작업 폴더 확인

[컨테이너]

```
mkdir -p /workspace/card4
ls -la /workspace
```

`/workspace/card4` 가 Windows 의 `C:\harness-lab\card4` 로도 보이면 연결이 된 것입니다. Windows 탐색기에서 확인합니다.

확인 체크리스트

- ■ `docker version` 에 Server 가 나온다 (Windows)
- ■ 컨테이너 안에서 `opencode --version` 이 나온다
- ■ 컨테이너 안에서 `opencode` → `/models` 목록이 나온다
- ■ Windows 의 `C:\harness-lab\card4` 가 보인다

자주 만나는 오류

증상	해결
`error during connect`	Docker Desktop 이 Engine running 이 아니다. 기다리거나 재시작한다.
`/workspace` 가 비어 있다	`.env` 의 `HARNESS_LAB_DIR` 이 틀렸다. `C:/harness-lab` 처럼 슬래시로 쓰고 폴더가 있는지 본다.
`git commit` 이 `Please tell me who you are`	`.env` 의 `GIT_NAME`, `GIT_EMAIL` 을 채우고 컨테이너를 다시 들어간다.
`/models` 목록이 비어 있다	키를 잘못 붙였거나 크레딧이 없다. OpenRouter 의 Keys, Credits 를 확인한다.

자세한 설명은 도커로 실행하기를 봅니다.

0단계 1장. AI 와 하네스 체험

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	1시간 30분
준비물	00 준비하기 완료
결과	같은 부탁을 조건 없이 한 것과 조건을 붙인 것을 비교한다

학습 목표

1. AI 모델과 AI 에이전트의 차이를 말한다.
2. "하네스"가 무엇인지 말로 설명한다.
3. 조건을 붙이면 결과가 달라지는 것을 내 눈으로 확인한다.

1. 개념

낱말	한 문장 설명	비유
AI 모델	글을 받아 글로 답한다. 내 파일은 보지 못한다.	전화로 대답하는 상담원
AI 에이전트	모델의 말대로 파일을 만들고 고친다. (이 교안에서는 OpenCode)	상담원의 지시로 일하는 직원
하네스	에이전트의 일을 통제하는 장치 전체	말에게 씌우는 마구

> **하네스(harness)** 는 말에게 씌우는 마구입니다. 말은 힘이 세지만 마구가 없으면 원하는 방향으로 달리지 못합니다. 마구는 힘을 줄이는 것이 아니라 쓸모 있게 만드는 것입니다.

하네스는 다섯 가지로 이루어집니다.

요소	질문	이 단계에서 만나는 것
규칙	어떻게 일해야 하나?	AGENTS.md (4장)
도구	무엇으로 일하나?	OpenCode, 브라우저
권한	어디까지 해도 되나?	opencode.jsonc (4장)
실행	어디에서 일하나?	작업 폴더 `/workspace`
확인	다 됐는지 어떻게 아나?	CHECKLIST.md (5장)

2. 따라 하기

1단계. 실험 A — 아무 조건 없이 시키기 (20분)

[컨테이너]

```
mkdir -p /workspace/card-a
cd /workspace/card-a
opencode
```

OpenCode 입력 칸에 한 줄만 씁니다.

■ [AI] OpenCode 입력 칸

> 내 자기소개 페이지를 HTML 로 만들어 줘.

AI 가 끝낼 때까지 기다린 뒤 `/exit` 로 나옵니다. 아래 표를 채웁니다.

관찰할 것	내 기록
계획을 먼저 말했나?	
파일을 몇 개 만들었나? 이름은?	
내 이름이나 취미를 물어봤나? 제멋대로 채웠나?	
그림, 외부 글꼴, 자바스크립트를 넣었나?	
완성이라고 했는데, 무엇으로 확인했나?	

[컨테이너]

```
ls -la
```

파일을 더블클릭해서 브라우저로 열어 봅니다.

2단계. 실험 B — 조건을 붙여서 시키기 (25분)

[컨테이너]

```
mkdir -p /workspace/card-b
cd /workspace/card-b
opencode
```

■ [AI] OpenCode 입력 칸

```
자기소개 페이지를 HTML 과 CSS 로 만들어 줘.
조건 1. 파일은 index.html 과 style.css 두 개만 만든다.
조건 2. 그림, 외부 글꼴, 자바스크립트는 쓰지 않는다.
조건 3. 내 이름과 취미를 묻고, 답을 받은 뒤에 만든다. 먼저 질문만 해 줘.
```

AI 가 질문하면 이름과 한 줄 소개, 취미 세 개를 답합니다. 끝난 뒤 실험 A 와 같은 표를 채우고, 아래 두 칸을 더 채웁니다.

항목	실험 A	실험 B
파일 개수		
자바스크립트나 외부 파일		
내 정보를 물었나?		

3단계. 차이를 말로 설명하기 (15분)

실험 B 의 조건 세 개가 각각 무엇을 막으려고 한 것인지 적습니다.

조건	막으려는 것
조건 1 (파일 두 개)	
조건 2 (외부 파일과 스크립트 금지)	
조건 3 (먼저 묻기)	

조건으로 적은 문장이 바로 **규칙(부탁)** 입니다. 4장에서 이런 문장을 `AGENTS.md` 에 옮깁니다.

3. 확인하기

- ■ 실험 A 와 B 의 차이를 두 가지 이상 적었다.
- ■ 조건 세 개가 각각 무엇을 막는지 말할 수 있다.
- ■ 하네스의 다섯 요소를 말할 수 있다.

4. 자주 만나는 오류

증상	해결
AI 가 "모델을 선택하세요"라고 한다	`/connect` 로 OpenRouter 키를 연결한다. (00 준비하기 5번)
브라우저에서 파일이 열리지 않는다	파일 이름이 `index.html` 인지 확인한다. 확장자가 `.txt` 로 바뀌었으면 고친다.
AI 가 계획 없이 바로 만든다	정상이다. 이 단계에서는 규칙이 없다. 조건 3 을 붙여 다시 시킨다.

5. 확인 문제

1. AI 모델과 AI 에이전트의 차이를 "파일"이라는 낱말로 설명하시오.
2. 실험 A 에서 AI 가 내 정보 없이 채운 것이 있다면, 그것이 왜 문제인가?
3. 하네스의 다섯 요소를 말하시오.

0단계 2장. 말로 된 요구사항 쓰기

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	1시간 30분
준비물	1장 완료
결과	명함 페이지의 요구사항을 "이렇게 하면 → 이런 결과" 형식으로 쓴다

학습 목표

1. 흐린 말과 확인할 수 있는 말을 구분한다.
2. 요구사항의 세 칸(이렇게 하면, 이런 결과, 잘못 쓰면)을 채운다.

1. 개념 — 요구사항은 AI 와의 약속

요구사항이 흐리면 AI 는 빈 곳을 제멋대로 채웁니다. 1장 실험 A 에서 내 이름 없이 페이지가 만들어진 것도 약속이 없었기 때문입니다.

흐린 말	확인할 수 있는 말
예쁘게 만든다	바탕은 연한 회색, 이름은 진한 남색이다
잘 보이게 한다	이름 글자가 가장 크고, 그 아래 소개가 보인다
모바일에서도 보기 좋게	창을 좁혀도 글이 가로로 잘리지 않고, 세로로 쌓인다
간단하게	파일은 두 개, 기능은 세 가지까지만

확인 방법을 적을 수 없는 말은 요구사항에 넣지 않습니다.

2. 따라 하기

1단계. 흐린 말 고치기 (15분)

아래 문장에서 흐린 낱말을 찾아 동그라미를 칩니다.

명함은 예쁘고, 보기 좋고, 누구나 쉽게 읽을 수 있어야 한다.

각각을 확인할 수 있는 말로 바꿉니다.

흐린 말	바꾼 말
예쁘고	
보기 좋고	
쉽게 읽을 수 있어야	

2단계. 명함 요구사항 쓰기 (30분)

저장소의 `templates/stage0/REQ_명함.md` 를 엽니다. 이미 쓰인 예시가 있으니 읽기만 합니다.

[Windows]

```
# (Windows) /tutorial/templates/stage0/REQ_명함.md 을(를) VS Code 로 열어 고칩니다
```

이번에는 **나만의 명함**을 씁니다. 같은 틀을 복사해서 채웁니다.

[컨테이너]

```
mkdir -p /workspace/명함요구사항
cp /tutorial/templates/stage0/REQ_명함.md /workspace/명함요구사항/내명함.md
# (Windows) /workspace/명함요구사항/내명함.md 을(를) VS Code 로 열어 고칩니다
```

조건은 다음과 같습니다.

- 기능은 **세 가지**만 씁니다. (이름과 소개, 취미 목록, 연락 링크)
- 각 기능에 "잘못 쓰면" 칸을 채웁니다.
- "만들지 않는 것"에 두 가지 이상 적습니다.

3단계. 점검하기 (15분)

■ 옆 사람이 읽고 같은 페이지를 만들 수 있는가?

■ 모든 기능에 "잘못 쓰면" 칸이 있는가?

■ 결과를 눈으로 확인할 수 있는 말만 있는가?

4단계. 옆 사람과 바꿔 읽기 (15분)

옆 사람의 요구사항을 읽고, 질문을 한 개씩 적습니다. 질문이 많으면 약속이 약하다는 뜻입니다.

예: "연락 링크를 누르면 이메일 창이 열리나요, 아니면 웹 페이지가 열리나요?" → 답이 적혀 있지 않으면 고칩니다.

3. 확인하기

■ 흐린 말을 세 개 이상 확인할 수 있는 말로 바꿨다.

■ 명함 요구사항에 세 기능과 "잘못 쓰면" 칸을 모두 채웠다.

■ 옆 사람에게서 질문을 하나 이상 받았다.

4. 자주 만나는 오류

증상	해결
"잘", "예쁘게"가 계속 나온다	1단계 표로 돌아간다.
"잘못 쓰면"을 무엇으로 쓸지 모르겠다	비울 때, 빈칸을 넣을 때, 너무 길 때를 하나씩 생각한다.

5. 확인 문제

1. "예쁘게 만든다"를 확인할 수 있는 말로 바꾸시오.
2. "잘못 쓰면" 칸이 비어 있으면 AI 에게 무슨 일이 생기는가?
3. 요구사항에 "자바스크립트로 만든다"를 적으면 안 되는 이유는 무엇인가?

0단계 3장. 작업 지시서 쓰기

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	1시간 30분
준비물	2장의 요구사항
결과	지시서 여섯 칸을 채워 AI 에게 건네고, 계획을 검토해 승인한다

학습 목표

1. 지시서의 여섯 칸을 말한다.
2. 한 줄 지시와 지시서 지시의 결과를 비교한다.
3. AI 의 계획을 검토하고 고치게 한다.

1. 개념 — 일을 나눠 주는 쪽지

칸	질문	빠지면 생기는 일
1. 목표	끝나면 무엇이 되어 있나?	어디까지가 끝인지 모른다
2. 요구사항	어느 번호를 만드나?	모양을 AI 가 지어낸다
3. 범위	어떤 파일을 고쳐도 되나?	관계없는 파일이 생긴다
4. 하지 말 것	무엇을 덧붙이면 안 되나?	시키지 않은 것이 생긴다
5. 완료 조건	끝났는지 무엇으로 확인하나?	"다 했습니다"를 믿을 수밖에 없다
6. 진행 방법	계획을 먼저 말하나?	방향이 틀려도 끝나고서야 안다

완료 조건은 **확인할 수 있는 말**로 씁니다. "잘 보인다"가 아니라 "CHECKLIST.md 의 2단계를 모두 체크한다"처럼 씁니다.

2. 따라 하기

1단계. 빈칸 틀 복사하기 (5분)

[컨테이너]

```
mkdir -p /workspace/지시서연습
cp /tutorial/templates/TASK_빈칸.md /workspace/지시서연습/지시서.md
# (Windows) /workspace/지시서연습/지시서.md 을(를) VS Code 로 열어 고칩니다
```

`TASK_빈칸.md` 는 1단계의 틀입니다. 0단계에서는 같은 여섯 칸을 쓰되, 아래 예시를 참고해 HTML 과 CSS 에 맞게 고칩니다.

2단계. 여섯 칸 채우기 (30분)

2장에서 쓴 명함 요구사항을 바탕으로 채웁니다. 예시는 다음과 같습니다.

칸	예시
1. 목표	명함 페이지를 파일 두 개(index.html, style.css)로 만든다
2. 요구사항	명함 요구사항 R1~R5
3. 범위	고쳐도 되는 파일: index.html, style.css / 고치면 안 되는 파일: CHECKLIST.md, AGENTS.md
4. 하지 말 것	자바스크립트, 그림 파일, 외부 글꼴, 요구사항에 없는 섹션
5. 완료 조건	CHECKLIST.md 의 항목을 모두 체크했다
6. 진행 방법	계획을 먼저 말하고, "진행" 전에는 파일을 만들지 않는다

3단계. 한 줄 지시와 비교하기 (20분)

1장 실험 A 와 이번 지시를 비교할 수 있도록, 새 폴더에서 지시서를 건넵니다.

[컨테이너]

```
mkdir -p /workspace/card3
cd /workspace/card3
opencode
```

■ [AI] OpenCode 입력 칸

```
/workspace/지시서연습/지시서.md 를 읽고 그 지시대로 작업해 줘. 먼저 계획만 말하고 내 승인을 기다려.
```

> 파일을 읽지 못하면 지시서 내용을 그대로 붙여 넣습니다.

4단계. 계획 검토하기 (20분)

AI 의 계획을 아래 질문으로 검토합니다.

- ■ 고칠 파일이 index.html 과 style.css 뿐인가?
- ■ 자바스크립트, 그림, 외부 글꼴을 쓰겠다는 말이 없는가?
- ■ CHECKLIST.md 를 고치겠다는 말이 없는가?
- ■ 연락 링크와 취미 목록을 계획에 넣었는가?

맞지 않으면 짚어서 고치게 합니다.

■ [AI] OpenCode 입력 칸

```
계획 2번에 그림 파일이 들어 있어. 지시서 4번에서 "하지 말 것"에 적은 것이라 빼고 다시 말해 줘.
```

맞으면 승인합니다.

■ [AI] OpenCode 입력 칸

```
진행
```

5단계. 결과 확인하기 (15분)

AI 가 끝나면 지시서 5번을 직접 확인합니다.

[컨테이너]

```
cd /workspace/card3
ls -la
```

- ■ 파일이 index.html 과 style.css 두 개뿐이다.
- ■ 두 파일을 더블클릭하면 브라우저에 페이지가 보인다.

6단계. 비교표 채우기 (10분)

항목	1장 실험 A (한 줄)	3장 (지시서)
계획을 먼저 말했나?		
시키지 않은 것이 있나?		
완료를 확인할 방법이 있었나?		

3. 확인하기

- ■ 지시서 여섯 칸을 모두 채웠다.
- ■ 계획에서 빠지거나 넘친 것을 하나 이상 고치게 했다.
- ■ 두 파일이 브라우저에서 열린다는 것을 내 눈으로 확인했다.

4. 좋은 지시와 나쁜 지시

나쁜 지시	좋은 지시
"명함 만들어 줘"	"지시서대로 index.html 과 style.css 를 만든다. 자바스크립트는 쓰지 않는다."
"디자인 이상해요, 고쳐 줘"	"바탕이 너무 밝다. 요구사항 R5 의 색으로 바꿔 줘. 다른 것은 바꾸지 마."
"알아서 잘 해 줘"	"두 방법이 있으면 장단점을 말하고 내가 고르게 해 줘."

5. 확인 문제

1. 지시서의 여섯 칸을 순서대로 말하시오.
2. 완료 조건을 "예쁘게 보이면 됨"으로 쓰면 어떤 문제가 생기는가?
3. "범위"와 "하지 말 것"의 차이를 설명하시오.

0단계 4장. 규칙과 권한 세우기

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	1시간 30분
준비물	3장 완료, `templates/stage0/AGENTS_빈칸.md`, `templates/stage0/opencode.jsonc`
결과	규칙 문서를 채우고, 권한이 막는 일과 묻는 일을 직접 확인한다

학습 목표

1. **규칙(부탁)** 과 **권한(잠금장치)** 의 차이를 말한다.
2. `AGENTS.md` 에 꼭 들어갈 문장을 채운다.
3. 권한 설정을 읽고, 막히는 일을 직접 확인한다.

1. 개념

	규칙 (AGENTS.md)	권한 (opencode.jsonc)
성격	부탁	잠금장치
작동	AI 가 읽고 따른다	금지된 일은 실행 자체가 막힌다
잊거나 어기면	어길 수 있다	막힌다

> 교실 문에 "들어가지 마시오"라고 써 붙이는 것이 규칙, 문을 잠그는 것이 권한입니다. 둘 다 있어야 합니다.

권한 한 줄 읽기

```
{ "action": "shell", "resource": "git commit *", "effect": "deny" }
```

칸	뜻
action	무엇을 하려는가 (shell: 명령, edit: 파일 고치기, read: 파일 읽기)
resource	무엇에 대해서인가 (`*` 는 아무 글자나)
effect	allow(허용) / ask(물어봄) / deny(금지)

규칙은 위에서 아래로 읽고, **가장 아래에 걸린 규칙이 이깁니다.**

2. 따라 하기

1단계. 작업 폴더에 설정 넣기 (10분)

[컨테이너]

```
mkdir -p /workspace/card4
cp /tutorial/templates/stage0/opencode.jsonc /workspace/card4/
cp /tutorial/templates/stage0/AGENTS_빈칸.md /workspace/card4/AGENTS.md
cp /tutorial/templates/stage0/CHECKLIST.md /workspace/card4/
cd /workspace/card4
```

```
ls -la
```

2단계. AGENTS.md 채우기 (30분)

[Windows]

```
# (Windows) AGENTS.md 을(를) VS Code 로 열어 고칩니다
```

[작성] 을 채웁니다.

절	무엇을 적나	예
1. 목표와 사용자	페이지의 목적과 보는 사람	자기소개 명함, 처음 만나는 사람
2. 파일 나누기	고쳐도 되는 파일	index.html, style.css 만 고친다
3. 일하는 순서	계획, 승인, 만들기, 확인, 보고	계획을 말하고 "진행"을 기다린다
4. 하지 말 것	저장, 체크리스트, 외부 파일	저장은 하지 않는다. CHECKLIST.md 는 고치지 않는다.
5. 쓰기 규칙	주석과 글꼴	두 파일 안의 모든 줄에 한국어 설명을 단다
6. 완료 확인	무엇을 확인하나	CHECKLIST.md 를 모두 체크한다
7. 보고 형식	보고에 들어갈 것	한 일, 바뀐 파일, 확인한 항목

꼭 들어가야 할 다섯 문장. 빠진 것이 있으면 추가합니다.

- 계획을 먼저 말하고, 사람의 "진행"을 기다린다는 문장
- CHECKLIST.md 를 고치지 않는다는 문장
- 저장은 하지 않는다는 문장
- 완료라고 말하기 전 체크리스트를 확인한다는 문장
- 바뀐 파일과 확인한 항목을 보고한다는 문장

3단계. 권한 설정 읽기 (15분)

[Windows]

```
# (Windows) opencode.jsonc 을(를) VS Code 로 열어 고칩니다
```

아래 질문에 답합니다.

1. AI 가 `CHECKLIST.md` 를 고치려 하면? (allow / ask / deny)
2. AI 가 `git commit` 을 실행하려 하면?
3. AI 가 `index.html` 을 고치려 하면? (힌트: 걸리는 `edit` 규칙이 있나?)
4. AI 가 인터넷에서 그림을 가져오려 하면?

> 정답: 1 deny, 2 deny, 3 걸리는 규칙이 없으니 허용, 4 deny

4단계. 막히는지 확인하기 (25분)

[컨테이너]

```
git init
opencode
```

OpenCode 에서 하나씩 시키고 결과를 적습니다.

■ [AI] OpenCode 입력 칸

```
CHECKLIST.md 의 첫 줄에 "확인함" 이라고 적어 줘.
```

■ [AI] OpenCode 입력 칸

```
git commit -m "test" 를 실행해 줘.
```

■ [AI] OpenCode 입력 칸

```
구글에서 명함 그림을 찾아서 내려받아 줘.
```

시킨 일	기대하는 결과	실제 결과
CHECKLIST.md 고치기	막힌다 (deny)	
git commit 실행	막힌다 (deny)	
인터넷에서 그림 찾기	막힌다 (deny)	

확인이 끝나면 `/exit` 로 나온 뒤 `git status` 로 파일이 바뀌지 않았는지 봅니다.

[컨테이너]

```
git status
```

3. 확인하기

■ AGENTS.md 에 꼭 들어갈 다섯 문장이 모두 있다.
■ 권한 설정의 세 규칙을 말로 설명할 수 있다.
■ 막혀야 할 일 세 가지가 실제로 막혔다.

4. 자주 만나는 오류

증상	해결
AI 가 AGENTS.md 를 읽지 않는다	파일이 `/workspace/card4` 맨 위에 있는지 확인하고 OpenCode 를 새로 연다.
막혀야 할 일이 막히지 않는다	설정 파일의 쉼표와 따옴표를 확인하고 OpenCode 를 다시 연다. 그래도 안 되면 강사에게 알린다.

5. 확인 문제

1. 규칙과 권한의 차이를 설명하시오.
2. 권한 설정에서 `"effect": "deny"` 와 `"effect": "ask"` 의 차이는 무엇인가?
3. AI 가 CHECKLIST.md 를 고칠 수 있다면, "확인 완료"라는 말을 믿을 수 있는가? 이유를 쓰시오.

0단계 5장. 체크리스트로 확인하고 저장하기

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	1시간 30분
준비물	4장의 `/workspace/card4` 폴더
결과	AI 의 말이 아니라 체크리스트로 "다 됐다"를 판단하고, 체크가 끝난 뒤에만 저장한다

학습 목표

1. AI 의 "완료했습니다"를 그대로 믿으면 안 되는 이유를 말한다.
2. 체크리스트를 사람이 직접 확인하는 방법을 익힌다.
3. 체크가 끝난 것만 저장한다.

1. 개념 — 말과 확인은 다르다

AI 는 "완료했습니다"라고 말할 수 있습니다. 하지만 그 말이 맞는지는 **사람이 확인**해야 합니다. 체크리스트는 그 확인을 빠뜨리지 않게 해 주는 목록입니다.

확인하는 방법	예	한계
AI 의 보고를 읽는다	"모든 기능을 구현했습니다"	맞는지 알 수 없다
체크리스트를 직접 본다	"좁은 화면에서 글이 잘리지 않는다"	내가 직접 눈으로 봐야 한다

체크리스트의 한 줄은 **내가 직접 보고 확인할 수 있는 것**이어야 합니다.

2. 따라 하기

1단계. AI 에게 명함 만들기 (30분)

[컨테이너]

```
cd /workspace/card4
opencode
```

■ [AI] OpenCode 입력 칸

```
TASK_명함.md 를 읽고 그 지시대로 작업해 줘. 먼저 계획만 말하고 내 승인을 기다려. 계획에 동의하면 "진행"이라고 할게.
```

`TASK_명함.md` 는 `templates/stage0` 에 있습니다. 이 폴더로 복사한 뒤 쓰거나, 3장의 지시서를 붙여 넣습니다.

계획을 확인하고 `진행` 이라고 답합니다. 끝나면 `/exit` 로 나옵니다.

2단계. 파일 확인하기 (5분)

[컨테이너]

```
ls -la
git status
```

■ 파일이 `index.html`, `style.css` 두 개뿐이다. (CHECKLIST.md 등 다른 파일이 바뀌지 않았다)

3단계. 체크리스트 1~2단계 (10분)

`CHECKLIST.md` 를 열고 1단계, 2단계 항목을 확인합니다. 브라우저에서 `index.html` 을 더블클릭해 엽니다.

[Windows]

```
# (Windows) CHECKLIST.md 을(를) VS Code 로 열어 고칩니다
# (Windows) index.html 을(를) 탐색기에서 더블클릭합니다
```

확인한 항목에만 `[x]` 를 표시합니다. 확인하지 않았는데 체크하면 안 됩니다.

4단계. 체크리스트 3~4단계 (15분)

브라우저 창에서 내용을 확인합니다.

- 취미 목록이 세 개 세로로 보이는지
- 이메일 링크를 눌렀을 때 작성 창이 열리는지
- 창의 폭을 좁혔을 때 글이 잘리지 않는지

창 폭 줄이기: 브라우저 창의 오른쪽 모서리를 잡고 천천히 왼쪽으로 줄입니다. 가로 스크롤바가 생기면 실패입니다.

확인한 항목을 체크합니다.

5단계. 실패한 항목을 고치게 하기 (20분)

체크되지 않은 항목이 있다면, 어떤 항목이 안 되는지 AI 에게 말합니다. 체크리스트 자체를 고치라고 하면 안 됩니다.

■ [AI] OpenCode 입력 칸

```
CHECKLIST.md 의 3단계 두 번째 항목이 안 돼. 창을 좁히면 이메일 링크가 화면 밖으로 나가. 원인을 먼저 설명하고, 내가 "진행"이라고 하면 sty
  le.css 만 고쳐 줘. CHECKLIST.md 는 고치지 마.
```

고친 뒤 4단계로 돌아가 다시 확인합니다.

6단계. 저장하기 (10분)

모든 항목에 체크가 끝나면 저장합니다. 저장은 **사람이** 합니다.

[컨테이너]

```
git add -A
git status
git commit -m "5장: 명함 페이지 완성 (체크리스트 확인 완료)"
git log --oneline
```

커밋 기록이 한 줄 생기면 끝입니다.

> `git status` 에서 **체크하지 않은 항목이 있는데** 저장하고 싶어진다면, 저장하지 말고 5단계로 돌아갑니다.

3. 확인하기

■ 체크리스트의 모든 항목을 직접 확인했다.

■ 실패한 항목이 있었다면, AI 에게 원인부터 설명하게 했다.

■ 커밋 기록이 한 줄 생겼다.

4. 자주 만나는 오류

증상	해결
`start index.html` 이 열리지 않는다	파일 이름과 확장자를 확인한다. VS Code 로 열어 `.html` 인지 본다.
`git commit` 이 `Please tell me who you are`	00 준비하기 6번의 `git config --global` 두 줄을 다시 실행한다.
AI 가 CHECKLIST.md 를 고친다	4장의 권한 설정을 확인한다. 원래 내용을 `templates/stage0/CHECKLIST.md` 와 비교해서 AI 에게 원래대로 되돌리게 한다.

5. 확인 문제

1. AI 가 "완료했습니다"라고 말했을 때 사람이 확인해야 하는 이유는 무엇인가?
2. 체크리스트의 한 줄이 "예쁘게 보인다"이면 왜 좋지 않은가?
3. 저장을 사람이 하는 이유는 무엇인가?

0단계 6장. 미니 프로젝트: 자기소개 명함 페이지

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	1~5장 완료
결과	명함 페이지를 AI 에게 맡기고, 체크리스트로 확인해 저장한다. 과정 기록을 쓴다.

학습 목표

1. 1~5장의 요소(요구사항, 지시서, 규칙, 권한, 체크리스트, 저장)를 한 프로젝트에 모두 쓴다.
2. 내 이름과 취미로 명함을 채운다.
3. 체크리스트가 통과한 것만 저장한다.

1. 개념 — 지금까지의 고리

```
① 요구사항  (REQUIREMENTS.md)          무엇을 만들지 약속한다        ← 2장
② 작업 지시서 (TASK_명함.md)             어디까지 맡길지 정한다        ← 3장
③ 규칙과 권한 (AGENTS.md, opencode.jsonc) AI 의 행동 범위를 정한다     ← 4장
④ 확인 (CHECKLIST.md)                  사람이 직접 본다              ← 5장
⑤ 저장 (git commit)                    확인이 끝난 것만 저장한다      ← 5장
```

사람이 하는 일은 요구사항의 내용(내 정보)을 채우고, 계획을 승인하고, 체크리스트를 직접 보고, 저장하는 것입니다. 파일은 AI 가 만듭니다.

2. 따라 하기

1단계. 작업 폴더 만들기 (10분)

[컨테이너]

```
mkdir -p /workspace/my-card
cd /workspace/my-card
T="/tutorial/templates/stage0"
cp "$T/opencode.jsonc" .
cp "$T/CHECKLIST.md" .
cp "$T/AGENTS_빈칸.md" AGENTS.md
cp "$T/REQ_명함.md" REQUIREMENTS.md
cp "$T/TASK_명함.md" TASK.md
git init
echo ".env" > .gitignore
ls -la
```

2단계. 내 정보 채우기 (15분)

`REQUIREMENTS.md` 의 **R1 과 R2** 에 내 정보를 적습니다. 요구사항의 틀은 그대로 두고, 내용만 바꿉니다.

[Windows]

```
# (Windows) REQUIREMENTS.md 을(를) VS Code 로 열어 고칩니다
```

R1 에는 이름과 한 줄 소개, R2 에는 취미 세 개를 적습니다. 그 밖의 요구사항은 그대로 둡니다.

3단계. AGENTS.md 채우기 (20분)

[Windows]

```
# (Windows) AGENTS.md 을(를) VS Code 로 열어 고칩니다
```

4장에서 쓴 것을 옮깁니다. 꼭 들어갈 다섯 문장이 있는지 확인합니다.

- ■ 계획을 먼저 말하고 "진행"을 기다린다.
- ■ CHECKLIST.md 를 고치지 않는다.
- ■ 저장은 하지 않는다.
- ■ 완료라고 말하기 전 체크리스트를 확인한다.
- ■ 바뀐 파일과 확인한 항목을 보고한다.

4단계. AI 에게 맡기기 (30분)

[컨테이너]

```
opencode
```

■ [AI] OpenCode 입력 칸

> TASK.md 를 읽고 그 지시대로 작업해 줘. REQUIREMENTS.md 의 내용을 따른다. AGENTS.md 의 작업 순서를 지켜서, 먼저 계획만 말하고 내 승인을 기다려.

계획을 검토합니다.

- ■ 고칠 파일이 index.html, style.css 뿐인가?
- ■ 자바스크립트, 그림, 외부 파일을 쓰겠다는 말이 없는가?
- ■ CHECKLIST.md 를 고치겠다는 말이 없는가?

맞으면 진행 이라고 답합니다.

5단계. 체크리스트로 확인하기 (30분)

AI 가 끝나면 `/exit` 로 나옵니다. `CHECKLIST.md` 를 열고 1단계부터 확인합니다.

[컨테이너]

```
ls -la
# (Windows) CHECKLIST.md 을(를) VS Code 로 열어 고칩니다
# (Windows) index.html 을(를) 탐색기에서 더블클릭합니다
```

- 내 이름이 가장 크게 보이는가? 소개와 취미가 맞는가?
- 이메일 링크를 누르면 작성 창이 열리는가?
- 창을 좁혀도 글이 잘리지 않는가?

내가 확인하지 않은 항목은 체크하지 않습니다.

실패한 항목이 있다면 AI 에게 말합니다.

■ [AI] OpenCode 입력 칸

> CHECKLIST.md 의 항목 중 실패한 것이 있어. 어떤 항목인지 내가 말할게. 원인을 먼저 설명하고, 내가 "진행"이라고 하면 index.html 과 style.css 만 고쳐 줘. CHECKLIST.md 는 고치지 마.

6단계. 저장하기 (10분)

모든 항목을 확인했다면 저장합니다.

[컨테이너]

```
git add -A
git status
git commit -m "6장: 나의 자기소개 명함 완성 (체크리스트 확인 완료)"
git log --oneline
```

7단계. 과정 기록 쓰기 (10분)

`기록.md` 를 만들고 아래를 채웁니다.

```
# 6장 기록

## 1. AI 에게 시킨 일 (한 문장)

## 2. AI 의 계획 중 고치게 한 것 (없으면 "없음")

## 3. 실패한 체크 항목과 고친 방법

## 4. 내가 이해한 것 세 문장
1.
2.
3.

## 5. 새로 알게 된 것 한 가지
```

[Windows]

```
# (Windows) 기록.md 을(를) VS Code 로 열어 고칩니다
```

3. 확인하기

- ■ 명함에 내 이름, 소개, 취미 세 개가 들어 있다.
- ■ 체크리스트의 모든 항목을 내가 직접 확인했다.
- ■ 저장 커밋이 한 줄 생겼다.
- ■ 기록에 설명 세 문장을 썼다.

4. 막혔을 때

- 페이지가 깨졌다면 CHECKLIST.md 의 해당 항목을 AI 에게 가리키며 원인부터 설명하게 합니다.
- AI 가 체크리스트를 고치려 하면 권한 설정(opencode.jsonc)이 막습니다. 막히지 않았다면 `git status` 로 확인해 되돌립니다.
- 정답 파일은 제공하지 않습니다. 요구사항과 체크리스트를 기준으로 AI 와 대화하며 맞춰 갑니다.

5. 확인 문제

1. 이번 장에서 사람이 한 판단을 세 가지 쓰시오.
2. 체크하지 않은 항목이 있는데 저장하면 어떤 일이 생기는가?
3. 1장 실험 A 와 이번 장을 비교했을 때 달라진 점을 두 가지 쓰시오.

다음 단계

이것으로 0단계가 끝납니다. 1단계에서는 같은 흐름을 **프로그래밍 언어(Python)** 와 **자동 테스트**로 옮겨 갑니다.

부록. 템플릿과 예제

수업에서 복사해 쓰는 파일을 그대로 실었다. 주석을 따라 읽는다.

A-1. 명함 요구사항 (2장)

```
# 요구사항: 자기소개 명함 페이지 (0단계 6장)

## 한 문장 요약
내 이름, 한 줄 소개, 취미, 연락 링크를 보여 주는 한 페이지짜리 웹 명함. 파일 두 개(index.html, style.css)로 만든다.

## 기능 요구사항

### R1. 이름과 한 줄 소개
- 이렇게 하면: 페이지를 브라우저로 연다.
- 이런 결과가 나온다: 맨 위에 이름이 크게 보이고, 그 아래에 한 줄 소개가 보인다.
- 잘못 쓰면: 이름이나 소개 칸을 비워도 페이지가 깨지지 않는다. 비어 있으면 그 줄을 숨긴다.

### R2. 취미 목록
- 이렇게 하면: 페이지를 연다.
- 이런 결과가 나온다: "취미"라는 제목 아래에 취미가 세 개 세로로 나열된다.
- 잘못 쓰면: 취미가 세 개가 아니어도 목록이 깨지지 않는다.

### R3. 연락 링크
- 이렇게 하면: "이메일 보내기" 링크를 누른다.
- 이런 결과가 나온다: 이메일 작성 창이 열린다.
- 잘못 쓰면: 링크 주소가 비어 있으면 링크를 보여 주지 않는다.

### R4. 좁은 화면에서도 보기 좋게
- 이렇게 하면: 브라우저 창을 휴대폰 폭처럼 좁힌다.
- 이런 결과가 나온다: 글과 목록이 가로로 잘리지 않고 세로로 쌓인다.
- 잘못 쓰면: 가로 스크롤바가 생기지 않는다.

### R5. 색과 글꼴
- 이렇게 하면: 페이지를 연다.
- 이런 결과가 나온다: 바탕은 연한 회색, 이름은 진한 남색이다. 글꼴 크기는 이름이 가장 크다.

## 만들지 않는 것
- 로그인, 방명록, 입력 폼
- 그림 파일 (글과 색만 쓴다)
- 자바스크립트 (HTML 과 CSS 만 쓴다)
```

A-2. 명함 작업 지시서 (3장·6장)

```
# 작업 지시서: 자기소개 명함 페이지 (0단계 6장)

## 1. 목표
`REQ_명함.md` 의 R1~R5 를 만족하는 명함 페이지를 HTML 파일 하나와 CSS 파일 하나로 만든다.

## 2. 요구사항
`REQUIREMENTS.md` (이 폴더에 복사해 둔 명함 요구사항)의 R1~R5.

## 3. 범위
- 새로 만드는 파일: `index.html`, `style.css` 두 개뿐이다.
- 고치면 안 되는 파일: `CHECKLIST.md`, `AGENTS.md`, `opencode.jsonc`

## 4. 하지 말 것
- 자바스크립트를 쓰지 않는다.
- 그림 파일이나 외부 글꼴, 외부 CSS 프레임워크(부트스트랩 등)를 가져오지 않는다.
- 요구사항에 없는 섹션(방명록, 입력 폼, 사진)을 덧붙이지 않는다.
- 파일 두 개 외에 다른 파일을 만들지 않는다.

## 5. 완료 조건
- [ ] `index.html` 과 `style.css` 두 파일이 있다.
- [ ] `CHECKLIST.md` 의 항목을 모두 확인했다. (사람이 확인한다)
- [ ] 두 파일 안에 한국어 주석이 있다. (무엇을 꾸미는지 한 줄씩)

## 6. 진행 방법
`AGENTS.md` 의 작업 순서를 따른다. 먼저 계획을 말하고, 내가 "진행"이라고 하면 만든다.
끝나면 무엇을 만들었고, `CHECKLIST.md` 의 어느 항목을 확인했는지 보고한다.
```

A-3. 규칙 문서 빈칸 틀 (4장)

```
# AGENTS.md — 작업 규칙 (빈칸을 채우는 틀)

> 4장에서 채웁니다. AI 에이전트(OpenCode)는 이 프로젝트에서 일을 시작할 때 이 파일을 먼저 읽습니다.
> 규칙은 "부탁"입니다. 옆의 권한 설정(`opencode.jsonc`)과 함께 있어야 힘이 생깁니다.
> `[작성]` 을 채우고, 아래 "꼭 들어가야 할 것"을 모두 체크합니다.

## 1. 이 프로젝트는 무엇인가
- 목표: [작성]
- 사용자: [작성] (이 페이지를 볼 사람)

## 2. 파일 나누기
| 파일 | 고쳐도 되는가 |
|---|---|
| index.html | [작성] |
| style.css | [작성] |
| CHECKLIST.md, AGENTS.md, opencode.jsonc | [작성] |

## 3. 일하는 순서
1. [작성] (먼저 무엇을 하는가)
2. [작성] (계획을 말하고 무엇을 기다리는가)
3. [작성] (만드는 방법)
4. [작성] (끝났는지 확인하는 방법)
5. [작성] (보고하는 방법)

## 4. 하지 말 것
- [작성] (커밋과 저장은 누가 하는가)
- [작성] (체크리스트에 대해 무엇을 금지하는가)
- [작성] (자바스크립트, 외부 파일, 새 파일에 대해)

## 5. 쓰기 규칙
- [작성] (주석은 어떤 언어로, 어디에 달 것인가)
- [작성] (글꼴과 색은 어떻게 정하는가)

## 6. 완료 확인
- 완료라고 말하기 전에 [작성] (무엇을 확인하는가. 체크리스트의 어느 부분인가)
- 확인이 안 되면 [작성] (무엇을 하는가)

## 7. 보고 형식
```

[한 일] [바뀐 파일] [확인한 체크리스트 항목] [확인 방법] [남은 일]

```
---

## 꼭 들어가야 할 것 (체크리스트)
- [ ] 계획을 먼저 말하고, 사람의 "진행"을 기다린다는 문장
- [ ] CHECKLIST.md 를 고치지 않는다는 문장
- [ ] 저장(commit)은 하지 않는다는 문장
- [ ] 완료라고 말하기 전 체크리스트를 확인한다는 문장
- [ ] 바뀐 파일과 확인한 항목을 보고한다는 문장
```

A-4. 권한 설정 (4장)

```
// ========================================================
// opencode.jsonc - 0단계 권한 설정 (HTML·CSS 프로젝트용)
//
// 4장에서 프로젝트 맨 위 폴더로 복사합니다.
// 규칙(AGENTS.md)은 "부탁", 이 파일은 "잠금장치"입니다.
//
// 읽는 법
//   action   : 무엇을 하려는가 (shell = 명령 실행, edit = 파일 고치기, read = 파일 읽기)
//   resource : 무엇에 대해서인가 (* 는 "아무 글자나")
//   effect   : allow(그냥 허용) / ask(나에게 묻기) / deny(금지)
//
// 위에서 아래로 읽으며, 여러 규칙에 걸리면 가장 아래 규칙이 이깁니다.
// ========================================================
{
  "$schema": "https://opencode.ai/config.json",
  "model": "openrouter/anthropic/claude-haiku-5.5",
  "share": "disabled",

  "permissions": [
    // ---------- 1. 명령 실행 ----------
    // 기본: 모든 명령은 실행 전에 나에게 묻는다.
    { "action": "shell", "resource": "*", "effect": "ask" },
    // 파일 목록과 Git 상태 확인은 묻지 않는다.
    { "action": "shell", "resource": "dir *", "effect": "allow" },
    { "action": "shell", "resource": "ls *", "effect": "allow" },
    { "action": "shell", "resource": "git status *", "effect": "allow" },
    { "action": "shell", "resource": "git diff *", "effect": "allow" },
    // 저장과 올리기는 사람만 한다.
    { "action": "shell", "resource": "git commit *", "effect": "deny" },
    { "action": "shell", "resource": "git push *", "effect": "deny" },
    { "action": "shell", "resource": "git reset *", "effect": "deny" },

    // ---------- 2. 파일 고치기 ----------
    // 규칙, 권한, 체크리스트(기준)는 AI 가 고치지 못한다.
    { "action": "edit", "resource": "*AGENTS.md", "effect": "deny" },
    { "action": "edit", "resource": "*opencode.jsonc", "effect": "deny" },
    { "action": "edit", "resource": "*CHECKLIST.md", "effect": "deny" },
    { "action": "edit", "resource": "*REQUIREMENTS.md", "effect": "deny" },

    // ---------- 3. 인터넷과 비밀 값 ----------
    { "action": "webfetch", "resource": "*", "effect": "deny" },
    { "action": "websearch", "resource": "*", "effect": "deny" },
    { "action": "read", "resource": "*.env", "effect": "deny" }
  ]
}
```

A-5. 확인 체크리스트 (5장)

```
# 검증 체크리스트 (명함 페이지)

> 이 파일이 "맞는지"를 판단하는 기준입니다. AI 는 이 파일을 고칠 수 없습니다.
> 완료라고 말하기 전에, 사람이 아래를 하나씩 직접 보고 체크합니다.
> 하나라도 빈칸이면 저장하지 않습니다.

## 1단계. 파일 확인 (1분)
- [ ] 폴더에 `index.html` 과 `style.css` 두 파일만 있다. (다른 파일이 생겼다면 AI 에게 되돌리게 한다)
- [ ] 두 파일을 메모장으로 열어도 글자가 깨져 보이지 않는다.

## 2단계. 브라우저로 열어 보기 (3분)
- [ ] `index.html` 을 더블클릭했을 때 브라우저에서 열린다.
- [ ] 페이지 맨 위에 이름이 가장 크게 보인다. (R1)
- [ ] 이름 아래에 한 줄 소개가 보인다. (R1)

## 3단계. 내용 확인 (3분)
- [ ] "취미" 제목 아래에 취미가 세 개 세로로 나열된다. (R2)
- [ ] "이메일 보내기" 링크를 누르면 이메일 작성 창이 열린다. (R3)
- [ ] 바탕은 연한 색, 이름은 진한 남색이다. (R5)

## 4단계. 좁은 화면 확인 (2분)
- [ ] 브라우저 창을 가장 좁게 줄여도 글이 가로로 잘리지 않는다. (R4)
- [ ] 가로 스크롤바가 생기지 않는다. (R4)
- [ ] 글과 취미 목록이 세로로 쌓여 보인다. (R4)

## 5단계. 코드 읽기 (5분)
- [ ] `index.html` 과 `style.css` 에 한국어 주석이 있다. (무엇을 꾸미는지 한 줄씩)
- [ ] 자바스크립트(`<script>`)가 없다.
- [ ] 외부 파일을 가져오는 줄(`http`, `https`)이 없다.

## 6단계. 저장 전 마지막 확인 (1분)
- [ ] 위의 모든 항목을 확인했다.
- [ ] 바꾼 파일과 확인 방법을 내가 말로 설명할 수 있다.

---

**체크를 다 했으면** 이 파일의 항목을 모두 체크한 상태로 저장합니다. 저장은 사람이 합니다. AI 는 저장하지 않습니다.
```